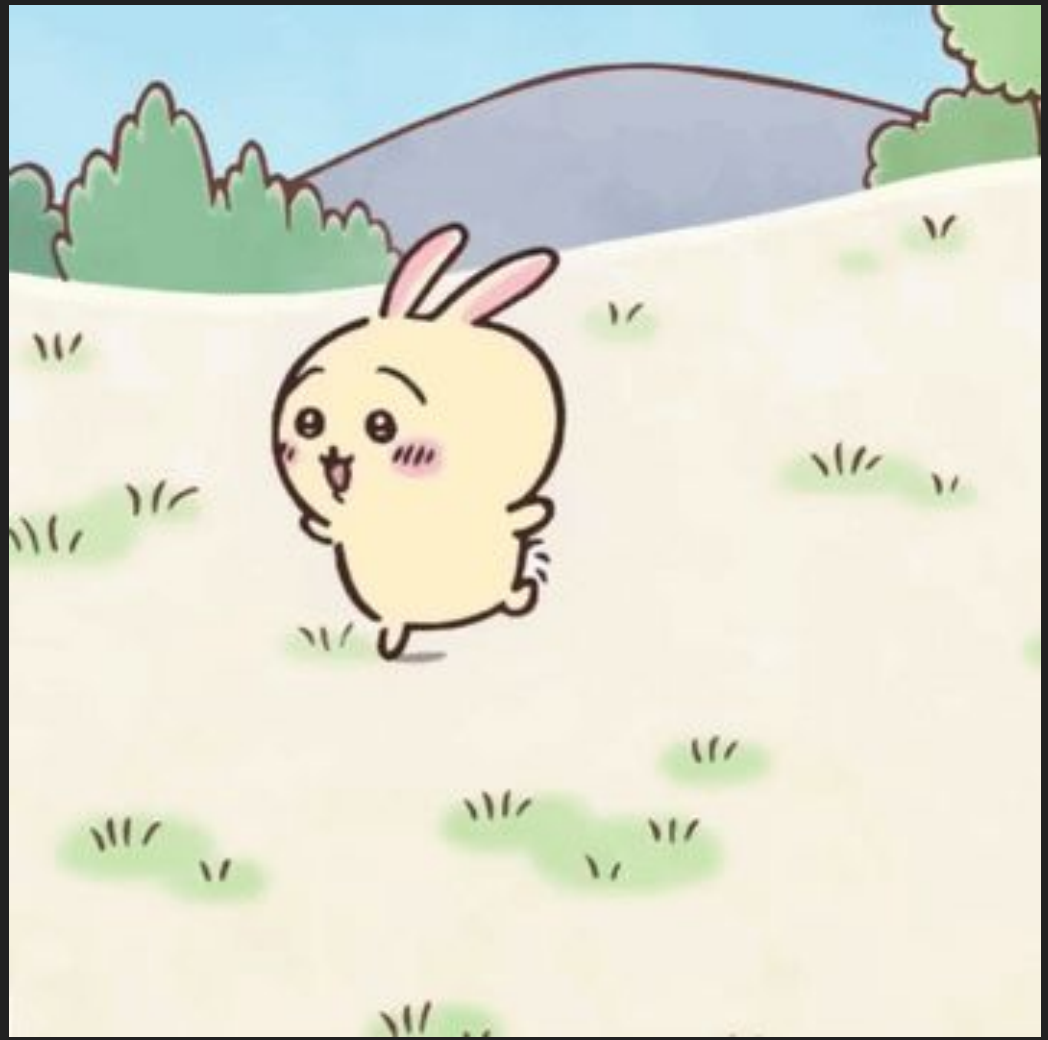
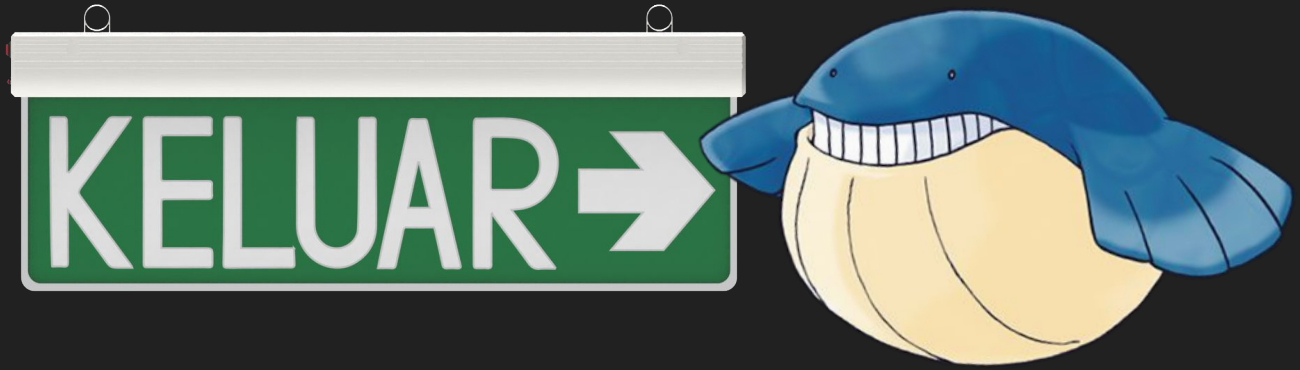


Container Escapes for Dummies

A gentle introduction to Docker container escapes



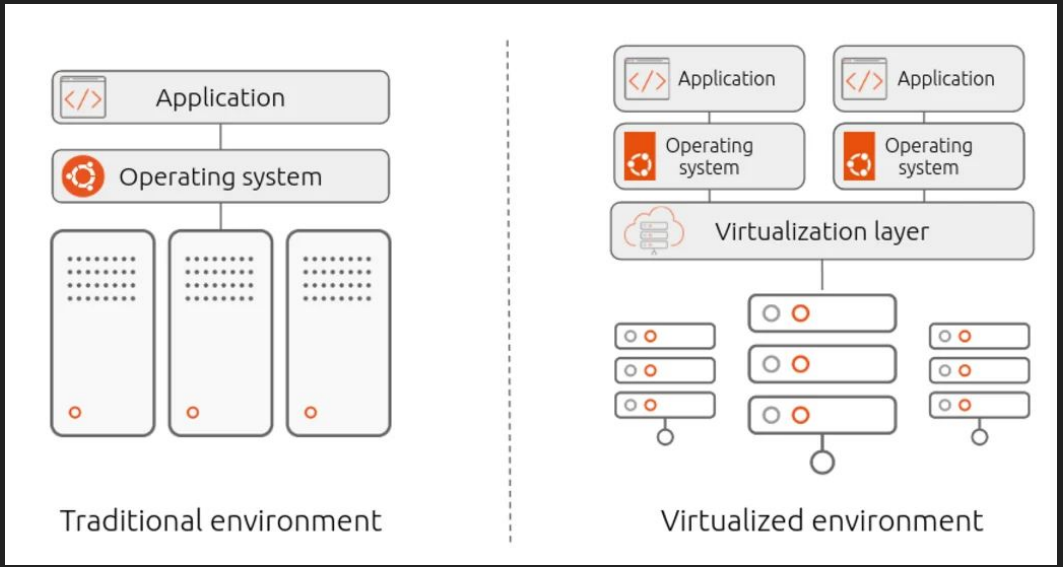


“container is just like using a VM”

- sakura nene (2026)

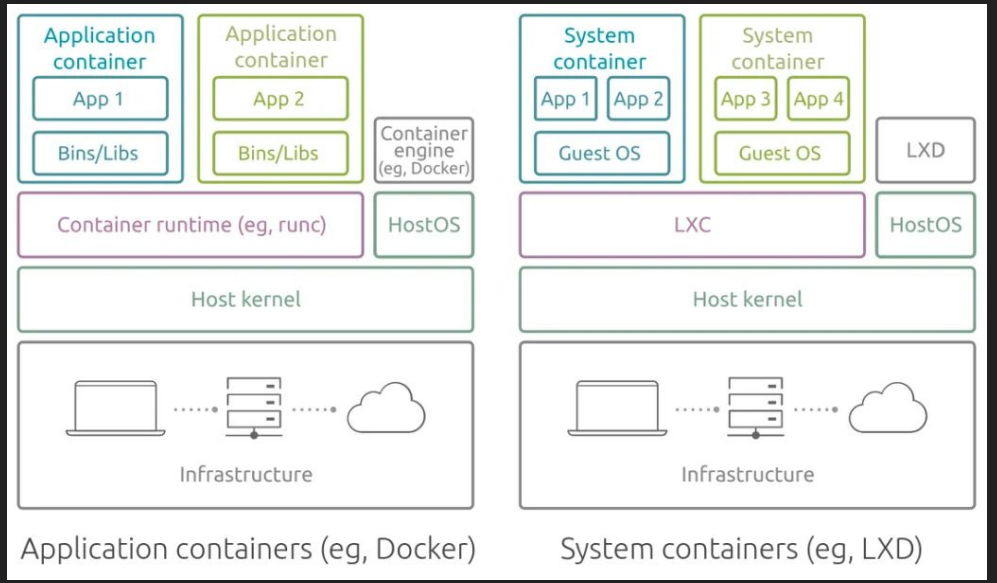
VMs and Containers

- Runs on top of a hypervisor
- VM image that includes the filesystem and state of the machine’s memory
- The hypervisor “emulates” hardware for the VM



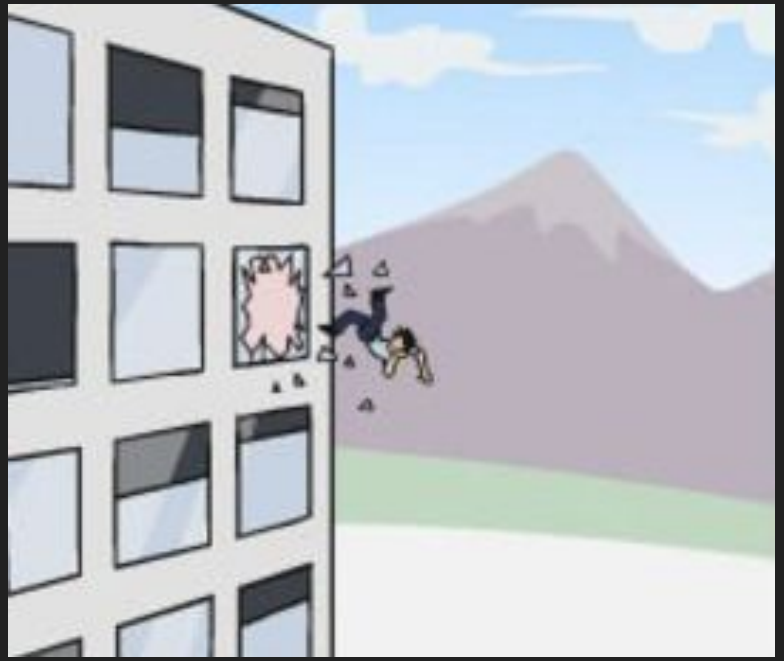
VMs and Containers

- Runs on a shared kernel
- A container is like a process
- Each process is “isolated” through Linux namespaces
- Makes it appear like a separate Linux machine



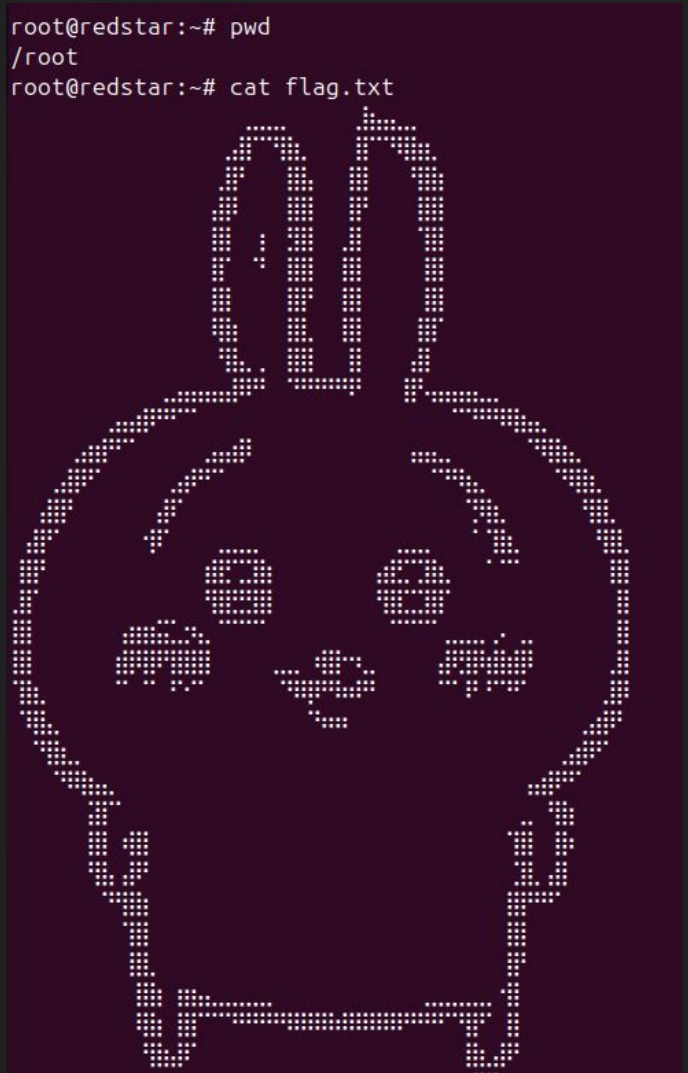
Container escapes

1. There is vulnerability in the container runtime or in the kernel that allows you to break namespace isolation.
2. Misconfigured containers.



Demos

For all the examples, the goal is to read flag.txt in my host’s /root directory.

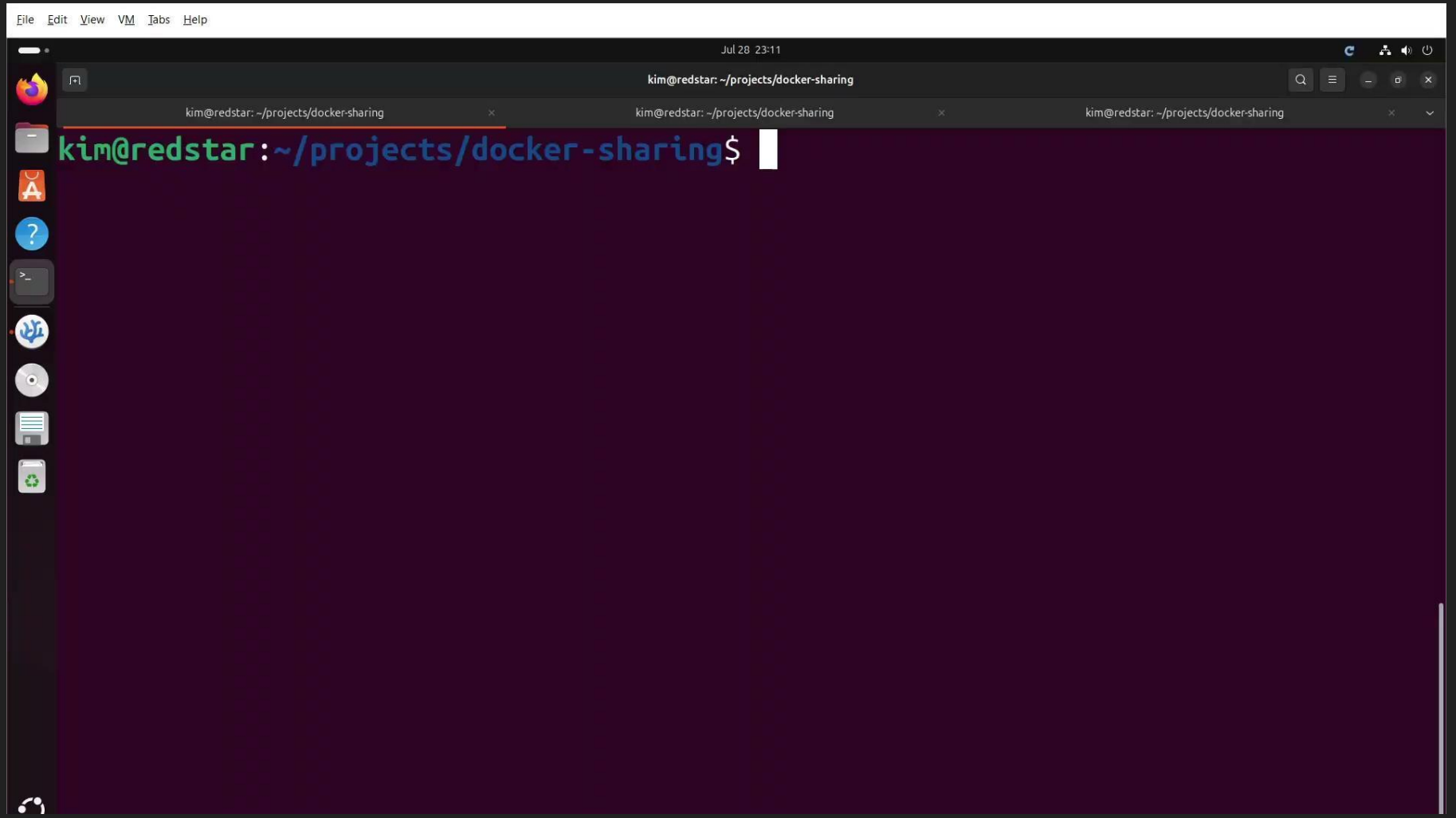


Example 1: Privileged containers

- God mode
- Containers start with a restricted set of capabilities (safe defaults)
- Privileged -> we are root on host :D




docker run --rm -it \
--cap-drop=ALL \
--privileged \
--name privilege-container \
alpine:3.23.5 \
sh


File Edit View VM Tabs Help
Jul 28 23:11
kim@redstar: ~/projects/docker-sharing
kim@redstar: ~/projects/docker-sharing
kim@redstar: ~/projects/docker-sharing
kim@redstar: ~/projects/docker-sharing
kim@redstar:~/projects/docker-sharing$

Example 2: Excessive capabilities

- Capabilities define what a process can do
- Gives you granular control over permissions instead of just granting root or having too much restrictions
- 40 capabilities in the kernel
- CAP_SYS_CHROOT, CAP_SYS_ADMIN, CAP_SYS_PTRACE, CAP_SYS_MODULE, blah blah blah…

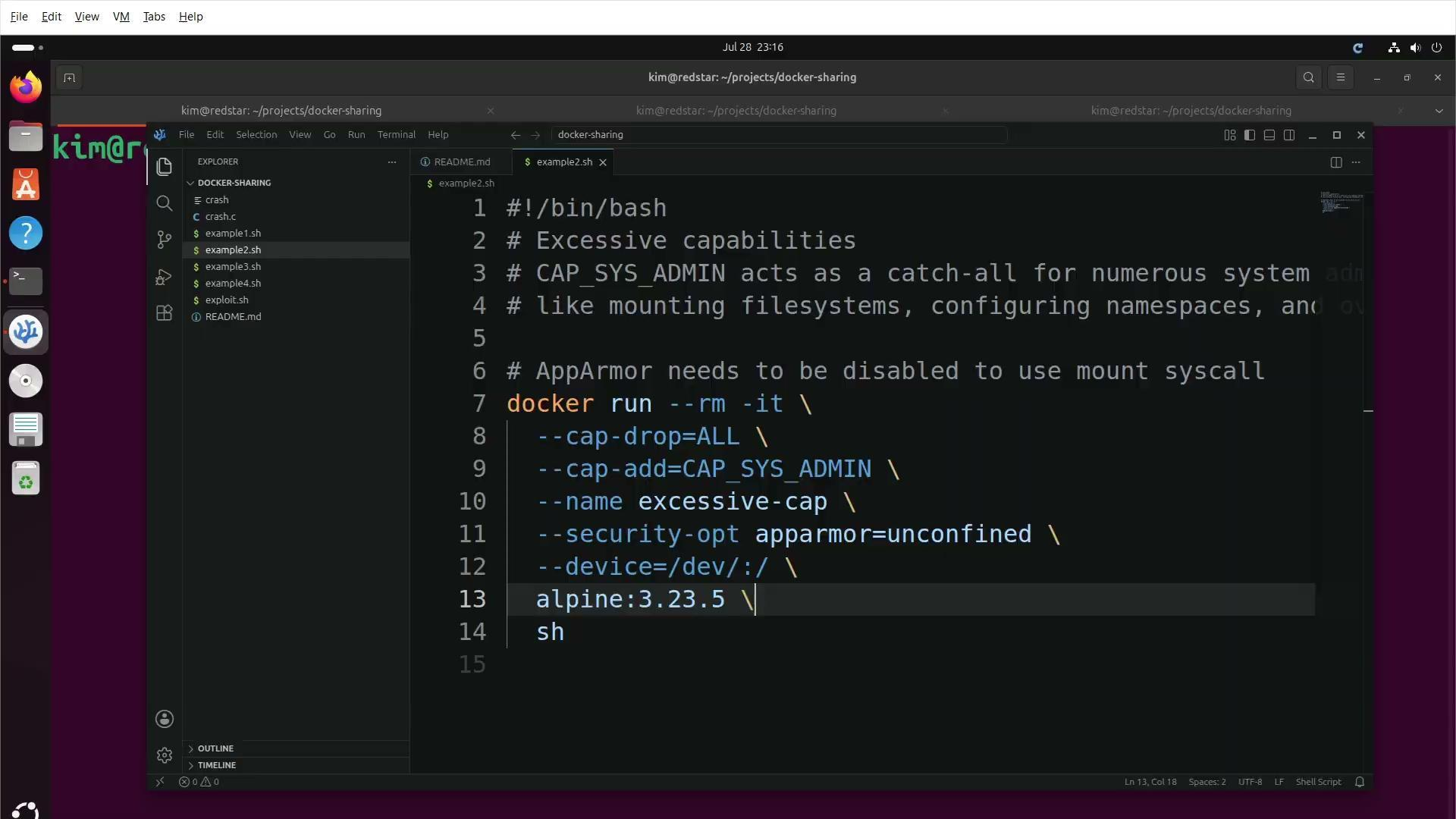


```
# if you have capsh in the container
$ capsh --print

# alternative
$ cat /proc/<proc_id>/status
CapInh: 0000000000000000
CapPrm: 000001ffffffffff
CapEff: 000001ffffffffff
CapBnd: 000001ffffffffff
CapAmb: 0000000000000000

# decode
$ capsh --decode=000001ffffffffff
0x000001ffffffffff=cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_fsetid,cap_kill,cap_setg
id,cap_setuid,cap_setpcap,cap_linux_immutable,cap_net_bind_service,cap_net_broadcast,cap_net_admin,cap_ne
t_raw,cap_ipc_lock,cap_ipc_owner,cap_sys_module,cap_sys_rawio,cap_sys_chroot,cap_sys_ptrace,cap_sys_pacct
,cap_sys_admin,cap_sys_boot,cap_sys_nice,cap_sys_resource,cap_sys_time,cap_sys_tty_config,cap_mknod,cap_l
ease,cap_audit_write,cap_audit_control,cap_setfcap,cap_mac_override,cap_mac_admin,cap_syslog,cap_wake_ala
rm,cap_block_suspend,cap_audit_read,cap_perfmon,cap_bpf,cap_checkpoint_restore
```


docker run --rm -it \
--cap-drop=ALL \
--cap-add=CAP_SYS_ADMIN \
--name excessive-cap \
--security-opt apparmor=unconfined \
--device=/dev/:/ \
alpine:3.23.5 \
sh


File Edit View VM Tabs Help
Jul 28 23:16
kim@redstar: ~/projects/docker-sharing
kim@redstar: ~/projects/docker-sharing
kim@redstar: ~/projects/docker-sharing
kim@redstar: ~/projects/docker-sharing
File Edit Selection View Go Run Terminal Help
docker-sharing
EXPLORER
DOCKER-SHARING
crash
crash.c
example1.sh
example2.sh
example3.sh
example4.sh
exploit.sh
README.md
README.md
example2.sh
example2.sh
#!/bin/bash
Excessive capabilities
CAP_SYS_ADMIN acts as a catch-all for numerous system
like mounting filesystems, configuring namespaces, and
AppArmor needs to be disabled to use mount syscall
docker run --rm -it \
--cap-drop=ALL \
--cap-add=CAP_SYS_ADMIN \
--name excessive-cap \
--security-opt apparmor=unconfined \
--device=/dev/:/ \
alpine:3.23.5 \
sh
OUTLINE
TIMELINE
Ln 13, Col 18
Spaces: 2
UTF-8
LF
Shell Script

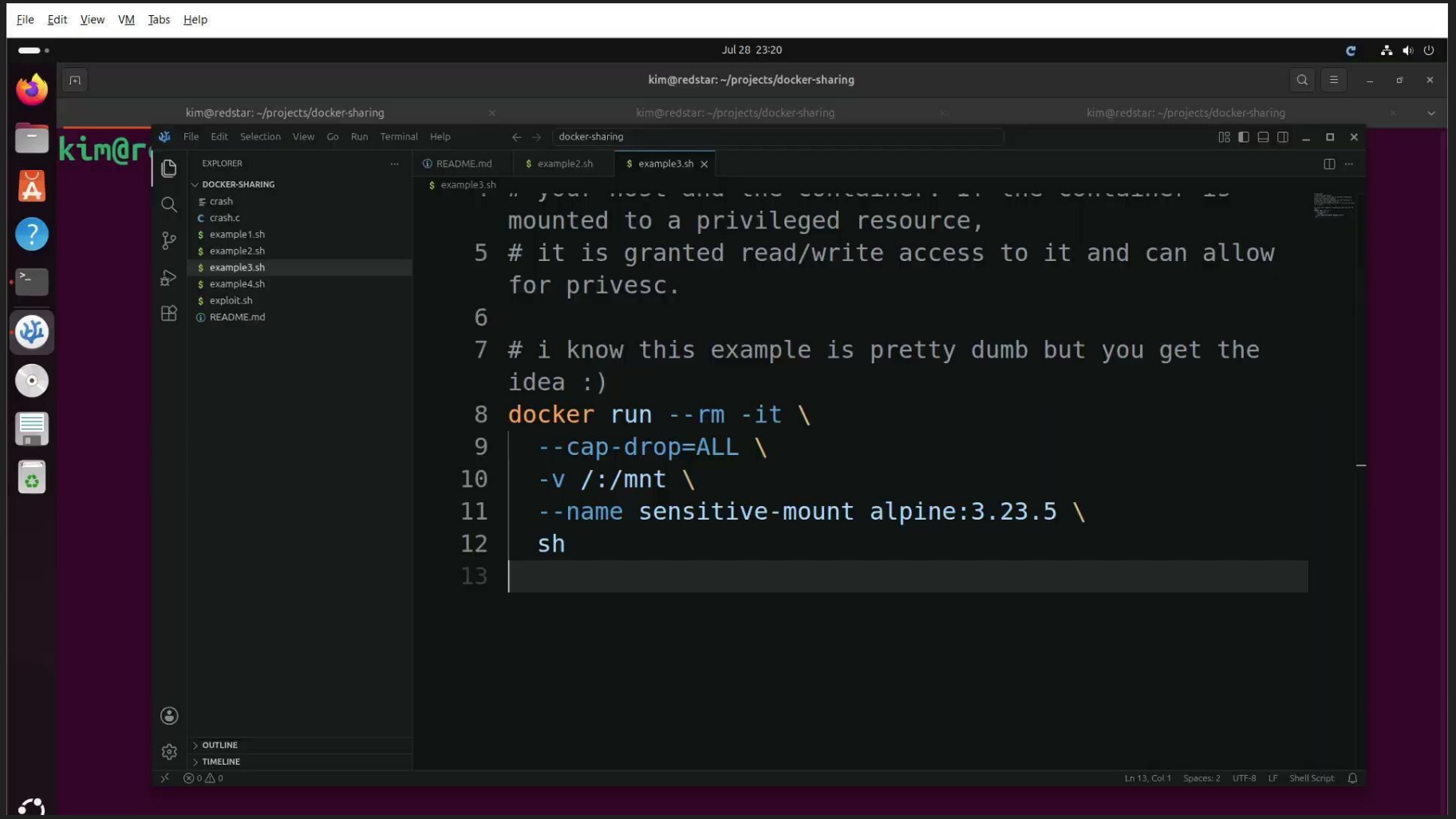
Example 3: Sensitive host mounts

- Sharing the host file system with your container
- The `-v` flag
- Escape/privesc depending on what you mount
- If the mount is sensitive/high privileged, the container is granted read and write access to it





```
docker run --rm -it \
  --cap-drop=ALL \
  -v /:/mnt \
  --name sensitive-mount alpine:3.23.5 \
  sh
```


File Edit View VM Tabs Help
Jul 28 23:20
kim@redstar: ~/projects/docker-sharing
File Edit Selection View Go Run Terminal Help
docker-sharing
EXPLORER
DOCKER-SHARING
crash
crash.c
example1.sh
example2.sh
example3.sh
example4.sh
exploit.sh
README.md
README.md
example2.sh
example3.sh
example3.sh
mounted to a privileged resource,
it is granted read/write access to it and can allow for privesc.
i know this example is pretty dumb but you get the idea :)
docker run --rm -it \
--cap-drop=ALL \
-v /:/mnt \
--name sensitive-mount alpine:3.23.5 \
sh
OUTLINE
TIMELINE
Ln 13, Col 1
Spaces: 2
UTF-8
LF
Shell Script

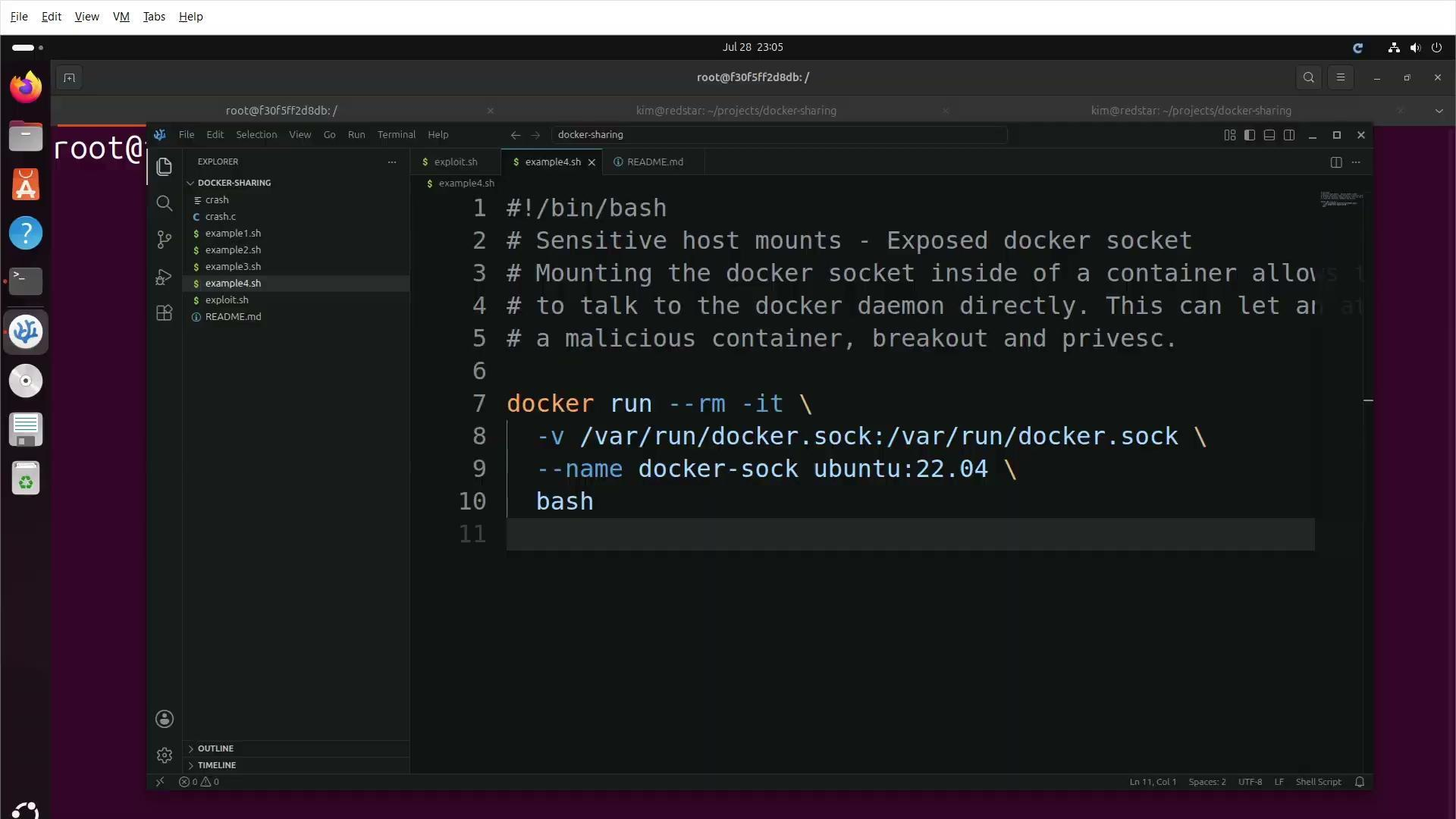
Example 4: Exposed docker socket

- Mounting docker socket inside of another container
- Containers inside of a container (???)
- Allows you to talk to the docker daemon directly
- It’s unauthenticated 😳





```
docker run --rm -it \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --name docker-sock ubuntu:22.04 \
  bash
```


File Edit View VM Tabs Help
Jul 28 23:05
root@f30f5ff2d8db: /
root@f30f5ff2d8db: /
kim@redstar: ~/projects/docker-sharing
kim@redstar: ~/projects/docker-sharing
File Edit Selection View Go Run Terminal Help
docker-sharing
EXPLORER
DOCKER-SHARING
crash
crash.c
example1.sh
example2.sh
example3.sh
example4.sh
exploit.sh
README.md
exploit.sh
example4.sh
README.md
#!/bin/bash
Sensitive host mounts - Exposed docker socket
Mounting the docker socket inside of a container allo
to talk to the docker daemon directly. This can let a
a malicious container, breakout and privesc.
docker run --rm -it \
-v /var/run/docker.sock:/var/run/docker.sock \
--name docker-sock ubuntu:22.04 \
bash
OUTLINE
TIMELINE
Ln 11, Col 1
Spaces: 2
UTF-8
LF
Shell Script

The end, goodnight

